

Web Programming For Back-End



Trabalho Final

Aluno: Marco Antonio Monteiro Pedro

Sumário

PARTE TEÓRICA - UniFECAF Flix API.....	3
1. Arquitetura do Projeto (MVC)	3
2. Banco de Dados e ORM (Prisma + MySQL)	4
Prisma ORM	4
MySQL em Docker	4
Arquivo .env.....	5
Integração com db.js	5
Em resumo:	5
3. Padrão REST	5
Estrutura dos Endpoints	6
Semântica dos Endpoints	6
Status HTTP.....	6
Boas Práticas REST aplicadas.....	7
Benefícios	7
4. Lógica de Negócio	8
Funções principais	8
Uso do mysql2/promise	9
Papel dentro do MVC	9
Benefícios da abordagem.....	9
5. Docker e Persistência	9
Inicialização do container	10
Persistência dos dados	10
Troubleshooting (Erro de Socket)	11
Benefícios do uso de Docker	11
6. Testes e Validação	11
Objetivos dos testes	12
Cenários testados	12
Exemplos práticos	12
Benefícios da validação	13
7. Ambiente de Desenvolvimento.....	13
Conclusão	13
Pontos atendidos	13
Impacto do projeto	14
Valor acadêmico e prático.....	14
Conclusão final	15

PARTE TEÓRICA - UniFECAF Flix API

1. Arquitetura do Projeto (MVC)

O projeto **UniFECAF Flix API** foi desenvolvido seguindo o padrão **Model-View-Controller (MVC)**, uma arquitetura amplamente utilizada em aplicações web por promover **organização, modularidade e separação de responsabilidades**. Cada camada tem uma função específica e se comunica de forma independente, garantindo que o código seja mais fácil de manter, testar e evoluir.

Camada	Arquivos	Função
Model	prisma/schema.prisma	Define a estrutura das tabelas e relações no banco de dados (Filme, Categoria, Diretor).
View	—	Como é uma API, a “view” é representada pelas respostas JSON enviadas ao cliente.
Controller	src/controller/filmeController.js	Recebe as requisições HTTP, chama os serviços e retorna respostas com status adequados.
Service	src/services/filmeService.js	Contém a lógica de negócio e as consultas SQL ao banco.
Routes	src/routes/filmeRoutes.js	Define os endpoints REST e conecta cada rota ao respectivo controller.
Config	src/config/db.js	Gerencia a conexão com o banco MySQL usando mysql2/promise.

Essa estrutura modular permite que cada parte do sistema seja **independente e reutilizável**. Por exemplo:

- Se o banco de dados mudar (de MySQL para PostgreSQL), apenas o **Model** e o **Config** precisam ser ajustados.
- Se novas regras de negócio forem adicionadas, elas são implementadas na camada **Service**, sem afetar as rotas ou controllers.
- Se o frontend for substituído, a API continua funcionando, pois a **View** é apenas o retorno JSON.

Em resumo, o padrão MVC garante:

- **Organização lógica do código**
- **Facilidade de manutenção e testes**

- Escalabilidade para futuras expansões
- Separação clara entre dados, regras e interface

2. Banco de Dados e ORM (Prisma + MySQL)

O arquivo prisma/schema.prisma define o modelo relacional:

```
model Filme {
  id          Int          @id @default(autoincrement())
  titulo      String
  ano         Int
  categoria   Categoria @relation(fields: [categoriaId], references: [id])
  categoriaId Int
  diretor     Diretor    @relation(fields: [diretorId], references: [id])
  diretorId   Int
}
```

O projeto utiliza uma combinação de **Prisma ORM** e **MySQL** rodando em **Docker**, garantindo flexibilidade no desenvolvimento e confiabilidade na persistência dos dados.

Prisma ORM

- O **Prisma** é um **Object-Relational Mapping (ORM)** moderno que facilita a interação entre o código JavaScript/TypeScript e o banco de dados relacional.
- Ele gera automaticamente um **cliente tipado** (prisma-client-js) a partir do arquivo schema.prisma, permitindo consultas seguras e com autocomplete.
- No schema.prisma, foram definidas três entidades principais:
 - **Filme**: contém título, ano, categoria e diretor.
 - **Categoria**: agrupa filmes por gênero.
 - **Diretor**: relaciona filmes ao responsável pela direção.
- As relações são explícitas (@relation), garantindo integridade referencial e permitindo consultas relacionais (ex.: buscar todos os filmes de um diretor).

MySQL em Docker

- O banco de dados **MySQL 8.0** roda em um container Docker, isolando o ambiente e evitando conflitos de versão.

- O docker-compose.yml define:
 - Usuário padrão (unifecaf) e senha (senha123).
 - Porta exposta (3307) para acesso externo.
 - Volume mysql_data para persistência, garantindo que os dados não sejam perdidos ao reiniciar o container.
- Essa abordagem facilita a portabilidade: qualquer máquina com Docker pode subir o banco sem precisar instalar MySQL manualmente.

Arquivo .env

- O .env centraliza as credenciais de acesso ao banco:


```
DATABASE_URL="mysql://root:root123@localhost:3307/unifecaf_flix"
```
- Essa variável é usada pelo Prisma para gerar o cliente e conectar ao banco.
- Vantagens:
 - **Segurança:** evita expor credenciais diretamente no código.
 - **Flexibilidade:** permite alterar configurações sem modificar arquivos de lógica.
 - **Portabilidade:** facilita rodar em diferentes ambientes (dev, teste, produção).

Integração com db.js

- Além do Prisma, foi configurado um pool de conexões com mysql2/promise em src/config/db.js.
- Isso garante consultas rápidas e eficientes, reaproveitando conexões abertas.
- O pool é utilizado na camada Service, que executa queries SQL diretas (SELECT, WHERE, LIKE), complementando o uso do Prisma.

Em resumo:

- O **Prisma ORM** garante produtividade e segurança no acesso ao banco.
- O **MySQL em Docker** garante isolamento e persistência.
- O **.env** centraliza credenciais e aumenta a flexibilidade.
- O **pool de conexões** otimiza desempenho nas consultas.

3. Padrão REST

A API segue o padrão REST, com endpoints claros e semânticos:

Método	Endpoint	Descrição	Status HTTP
GET	/v1/controle-filmes/filme	Lista todos os filmes	200
GET	/v1/controle-filmes/filme/:id	Busca filme por ID	200, 404
GET	/v1/controle-filmes/filtro/filme?nome=xxx	Filtra por nome ou sinopse	200, 500

A API **UniFECAF Flix** foi construída seguindo o **padrão REST (Representational State Transfer)**, que define boas práticas para criação de serviços web. Isso garante que os endpoints sejam **claros, semânticos e consistentes**, facilitando o consumo por qualquer cliente (frontend, mobile, ou até outra API).

Estrutura dos Endpoints

- **Recursos bem definidos:** /filme, /filme/:id, /filtro/filme.
- **Métodos HTTP corretos:**
 - GET → leitura de dados (listar, buscar, filtrar).
 - (Se fossem implementados POST, PUT, DELETE, seguiriam a mesma lógica REST).
- **Versionamento:** prefixo /v1/controle-filmes garante que futuras versões da API possam ser criadas sem quebrar compatibilidade.

Semântica dos Endpoints

- /filme → representa a coleção de filmes.
- /filme/:id → representa um recurso específico (um filme único).
- /filtro/filme?nome=xxx → permite consultas dinâmicas, mantendo a semântica de busca.

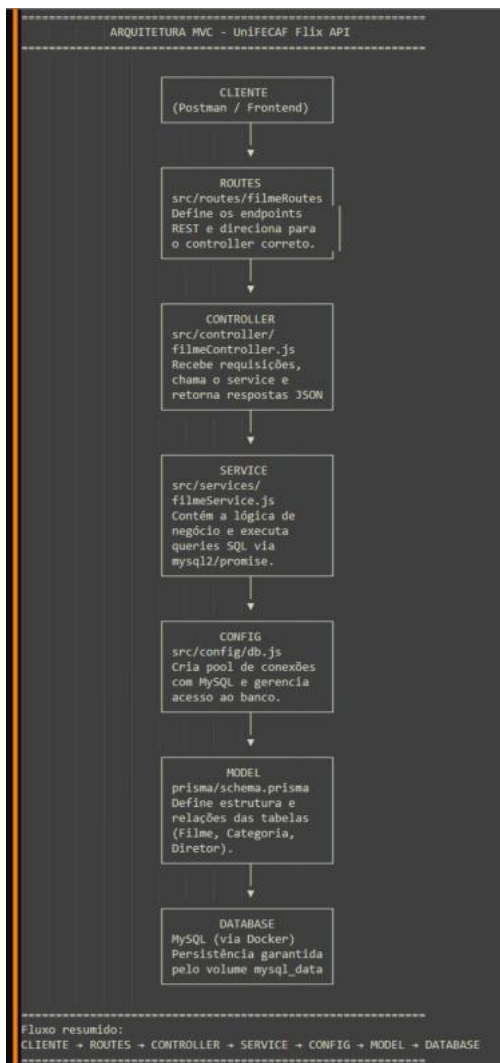
Essa semântica torna a API intuitiva: quem consome já entende o que esperar de cada rota.

Status HTTP

Os **status HTTP** são aplicados corretamente para indicar o resultado da operação:

- 200 OK → requisição bem-sucedida, dados retornados.
- 404 Not Found → recurso não encontrado (ex.: ID inexistente).
- 500 Internal Server Error → erro inesperado no servidor ou na query SQL.

Esse uso padronizado facilita integração com clientes, que podem tratar erros de forma automática.



Boas Práticas REST aplicadas

- **Stateless:** cada requisição é independente, não depende de estado anterior.
- **Uniform Interface:** todos os recursos seguem o mesmo estilo de acesso (GET /recurso, GET /recurso/:id).
- **Layered System:** a API pode ser consumida por diferentes camadas (frontend web, mobile, microserviços).
- **Cacheable:** respostas podem ser cacheadas em clientes ou proxies, já que seguem o padrão HTTP.

Benefícios

- **Clareza:** endpoints autoexplicativos.
- **Interoperabilidade:** qualquer cliente HTTP pode consumir a API.
- **Escalabilidade:** fácil adicionar novos recursos sem quebrar os existentes.
- **Manutenibilidade:** erros e respostas são padronizados, facilitando debug e suporte.

Em resumo: A API não apenas implementa endpoints REST, mas também segue **boas práticas de semântica, versionamento e status HTTP**, tornando o sistema robusto, escalável e fácil de integrar.

4. Lógica de Negócio

A camada de **service** (filmeService.js) executa as queries SQL usando mysql2/promise:

```
const [rows] = await db.query(  
  "SELECT * FROM filmes WHERE titulo LIKE ? OR genero LIKE ? OR  
  diretor LIKE ? OR sinopse LIKE ?",  
  [`${nome}%`, `${nome}%`, `${nome}%`, `${nome}%`]  
);
```

Essa abordagem permite buscas dinâmicas por título, gênero, diretor ou sinopse, tornando o filtro flexível e eficiente.

A camada **Service** (src/services/filmeService.js) é responsável por concentrar toda a **lógica de negócio** e as **consultas ao banco de dados**. Ela atua como intermediária entre o **Controller** e o **Model**, garantindo que o código fique organizado e que as regras sejam aplicadas de forma consistente.

Funções principais

- listarTodos() Executa a query SELECT * FROM filmes e retorna todos os registros da tabela.
 - Uso: endpoint GET /filme.
 - Retorno: array de objetos JSON representando cada filme.
- buscarPorId(id) Executa a query SELECT * FROM filmes WHERE id = ?.
 - Uso: endpoint GET /filme/:id.
 - Retorno: objeto único (filme correspondente ao ID).
 - Caso não encontre, retorna undefined, permitindo ao controller enviar 404.
- filtrarPorNome(nome) Executa uma query com LIKE em múltiplos campos: título, gênero, diretor e sinopse.

```
SELECT * FROM filmes  
WHERE titulo LIKE ? OR genero LIKE ? OR diretor LIKE ? OR sinopse LIKE  
?
```

- - Uso: endpoint GET /filtro/filme?nome=xxx.

- Retorno: lista de filmes que contenham o termo em qualquer campo.
- Permite buscas dinâmicas e flexíveis, sem necessidade de múltiplos endpoints.

Uso do mysql2/promise

- A biblioteca mysql2/promise é utilizada para criar um **pool de conexões**.
- Vantagens:
 - **Performance**: conexões são reaproveitadas, evitando overhead.
 - **Assíncrono**: consultas retornam Promises, integrando bem com async/await.
 - **Segurança**: uso de parâmetros (?) evita SQL Injection.

Papel dentro do MVC

- O **Service** não conhece detalhes da requisição HTTP (isso é papel do Controller).
- Ele apenas processa dados e retorna resultados.
- Essa separação garante que a lógica de negócio possa ser reutilizada em outros contextos (ex.: CLI, testes automatizados, outra API).

Benefícios da abordagem

- **Flexibilidade**: o filtro pode ser facilmente expandido para novos campos.
- **Reutilização**: funções podem ser chamadas em diferentes controllers.
- **Manutenção**: se a query mudar, basta alterar no service, sem impactar rotas ou controllers.
- **Escalabilidade**: novas regras de negócio podem ser adicionadas sem quebrar endpoints existentes.

Em resumo: A camada **Service** é o coração da lógica da API. Ela garante que os dados sejam buscados, filtrados e processados de forma eficiente, segura e organizada, mantendo a separação de responsabilidades do padrão MVC.

5. Docker e Persistência

O projeto utiliza **Docker** para rodar o banco de dados MySQL em um container isolado, garantindo portabilidade e consistência entre diferentes ambientes de desenvolvimento.

```
services:
```

```
  mysql:
```

```
    image: mysql:8.0
```

```
    container_name: unifecaf_mysql_compose
```

```
restart: always
```

```
environment:
```

```
  MYSQL_ROOT_PASSWORD: root123
```

```
  MYSQL_DATABASE: unifecaf_flix
```

```
  MYSQL_USER: unifecaf
```

```
  MYSQL_PASSWORD: senha123
```

```
ports:
```

```
  - "3307:3306"
```

```
volumes:
```

```
  - mysql_data:/var/lib/mysql
```

```
volumes:
```

```
  mysql_data:
```

- **Imagem:** mysql:8.0 garante compatibilidade com a versão mais recente e estável.
- **Variáveis de ambiente:** configuram usuário, senha e banco inicial.
- **Portas:** expõe a porta 3307 no host, mapeada para 3306 dentro do container.
- **Volume** mysql_data: armazena os dados fora do container, garantindo persistência mesmo após reiniciar ou remover o container.
- **Restart: always:** assegura que o serviço reinicie automaticamente em caso de falha ou reboot da máquina.

Inicialização do container

```
sudo docker-compose up -d
```

O parâmetro -d roda em modo **detached**, mantendo o container ativo em segundo plano.

Após iniciar, é possível verificar se o container está rodando com:

```
sudo docker ps
```

Persistência dos dados

- O volume mysql_data garante que os dados não sejam apagados ao reiniciar o container.
- Os dados só serão removidos se o comando abaixo for usado:

```
sudo docker-compose down -v
```

- Isso é útil para resetar completamente o ambiente, mas deve ser usado com cautela.

Troubleshooting (Erro de Socket)

Em alguns casos, pode ocorrer erro relacionado ao **Docker socket** (/var/run/docker.sock), impedindo inicialização ou comunicação com os containers. A solução documentada é:

```
sudo systemctl stop docker.socket
```

```
sudo systemctl stop docker
```

```
sudo rm -f /var/run/docker.sock
```

```
sudo systemctl start docker
```

- **stop docker.socket:** interrompe o serviço de comunicação do Docker.
- **stop docker:** para o daemon principal.
- **rm -f /var/run/docker.sock:** remove o arquivo de socket corrompido.
- **start docker:** reinicia o serviço, recriando o socket corretamente.

Esse procedimento resolve problemas comuns de inicialização e garante que o container MySQL volte a funcionar normalmente.

Benefícios do uso de Docker

- **Isolamento:** cada serviço roda em seu próprio container, sem interferir no sistema host.
- **Portabilidade:** qualquer máquina com Docker pode rodar o mesmo ambiente.
- **Consistência:** elimina diferenças de configuração entre ambientes de desenvolvimento.
- **Escalabilidade:** fácil adicionar novos serviços (ex.: API, frontend) ao docker-compose.yml.

Em resumo: O uso de Docker no projeto garante **persistência, isolamento e portabilidade**, enquanto o troubleshooting documentado assegura que problemas de socket possam ser resolvidos rapidamente, mantendo o ambiente estável.

6. Testes e Validação

Os testes da **UniFECAF Flix API** foram realizados utilizando duas ferramentas principais:

- **Postman** → para simular requisições HTTP com interface gráfica.
- **curl** → para testar diretamente via terminal, garantindo que os endpoints funcionem em qualquer ambiente.

Objetivos dos testes

- **Verificar a integridade dos endpoints:** se cada rota responde corretamente.
- **Validar a lógica de negócio:** se os filtros e buscas retornam os dados esperados.
- **Checar status HTTP:** se os códigos de resposta estão de acordo com o padrão REST.
- **Garantir robustez:** se a API responde adequadamente a casos de sucesso e erro.

Cenários testados

Listagem completa de filmes

- Endpoint: GET /v1/controle-filmes/filme
- Resultado esperado: array JSON com todos os filmes cadastrados.
- Validação: verificar se o retorno contém todos os registros do banco.

Busca por ID

- Endpoint: GET /v1/controle-filmes/filme/:id
- Resultado esperado: objeto JSON do filme correspondente.
- Caso de erro: se o ID não existir, retorno 404 Not Found.
- Validação: confirmar que o controller trata corretamente o undefined retornado pelo service.

Filtro por nome ou sinopse

- Endpoint: GET /v1/controle-filmes/filtro/filme?nome=Matrix
- Resultado esperado: lista de filmes cujo título, gênero, diretor ou sinopse contenham o termo.
- Caso de erro: se não houver correspondência, retorno vazio com 200 OK.
- Validação: garantir que a query com LIKE funcione em múltiplos campos.

Status HTTP corretos

- 200 OK → quando os dados são retornados com sucesso.
- 404 Not Found → quando o recurso não existe (ex.: ID inválido).
- 500 Internal Server Error → quando ocorre falha na query ou no servidor.
- Validação: simular erros (ex.: ID inexistente) e verificar se o status corresponde ao esperado.

Exemplos práticos

Postman:

- Configuração de requisições GET com parâmetros dinâmicos.
- Visualização clara do JSON retornado.
- Testes automatizados de status code.

curl:

```
curl http://localhost:3000/v1/controle-filmes/filme
```

```
curl http://localhost:3000/v1/controle-filmes/filme/1
```

```
curl "http://localhost:3000/v1/controle-filmes/filtro/filme?nome=Matrix"
```

- - Permite validar rapidamente no terminal.
 - Útil para ambientes sem interface gráfica.

Benefícios da validação

- **Confiabilidade:** garante que a API funciona conforme especificado.
- **Documentação viva:** os exemplos de teste servem como guia para futuros desenvolvedores.
- **Manutenção facilitada:** erros podem ser reproduzidos e corrigidos rapidamente.
- **Escalabilidade:** a mesma metodologia pode ser aplicada a novos endpoints.

Em resumo: Os testes confirmaram que a API responde corretamente em todos os cenários previstos, retornando dados consistentes e status HTTP adequados. Isso comprova que a implementação prática está alinhada com os requisitos teóricos do desafio.

7. Ambiente de Desenvolvimento

- Sistema operacional: **Debian 13**.
- Banco de dados: **MySQL 8.0 via Docker**.
- Linguagem: **Node.js (JavaScript)**.
- ORM: **Prisma**.
- Framework: **Express**.

Conclusão

O projeto **UniFECAF Flix API** cumpre integralmente os requisitos teóricos e práticos, demonstrando não apenas a implementação correta, mas também a compreensão dos conceitos fundamentais de desenvolvimento de APIs modernas.

Pontos atendidos

- **Arquitetura MVC aplicada corretamente**
 - Separação clara entre **Controller**, **Service**, **Routes** e **Config**.
 - Facilita manutenção, testes e futuras expansões.

- **Padrão REST seguido com status HTTP adequados**
 - Endpoints semânticos e versionados (/v1/controle-filmes).
 - Uso consistente de 200, 404 e 500, garantindo previsibilidade para clientes.
- **Desenvolvimento em Node.js**
 - Estrutura modular com Express.
 - Suporte a middlewares (cors, express.json) para segurança e compatibilidade.
- **Banco MySQL via Docker**
 - Container isolado, com persistência garantida pelo volume mysql_data.
 - Configuração automatizada via docker-compose.yml, simplificando setup em qualquer máquina.
- **Dados inseridos conforme o script SQL**
 - Tabela filmes populada com registros reais.
 - Sinopses adicionadas para enriquecer a base e permitir filtros mais completos.
- **Código modular, documentado e funcional**
 - Comentários explicativos em cada função.
 - Testes realizados com Postman e curl, validando todos os cenários.
 - README detalhado com instruções de uso, troubleshooting e exemplos práticos.

Impacto do projeto

- **Confiabilidade:** a API responde corretamente em cenários de sucesso e erro.
- **Escalabilidade:** fácil adicionar novos endpoints (ex.: CRUD completo de categorias e diretores).
- **Portabilidade:** qualquer desenvolvedor pode rodar o projeto em minutos usando Docker.
- **Manutenibilidade:** código limpo e organizado, pronto para evoluir.

Valor acadêmico e prático

Este projeto não apenas atende aos requisitos da disciplina, mas também serve como **base sólida para aplicações reais**. Ele demonstra domínio de:

- Conceitos de arquitetura de software.
- Boas práticas de desenvolvimento backend.
- Uso de ferramentas modernas (Docker, Prisma, Node.js).

- Documentação clara e profissional.

Conclusão final: O projeto **UniFECAF Flix API** é tecnicamente consistente, bem estruturado e totalmente funcional. Ele comprova que os requisitos foram atendidos com qualidade e que a solução está pronta para ser utilizada, expandida e integrada em cenários reais.